INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Persistencia de datos con Room Database en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	09	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	27/10/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca, Sergio Danilo				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Mg Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Link Propuesta: https://github.com/shanccom/IDNP_Repo/tree/main/Lab_09_PersistenciaRoom_Compose

Ejercicio resuelto por el docente

Paso 1. Crear un nuevo proyecto

nombre sugerido para el proyecto: “PersistenciaRoom_Compose”, Package name: com.example.persistenciaroom_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

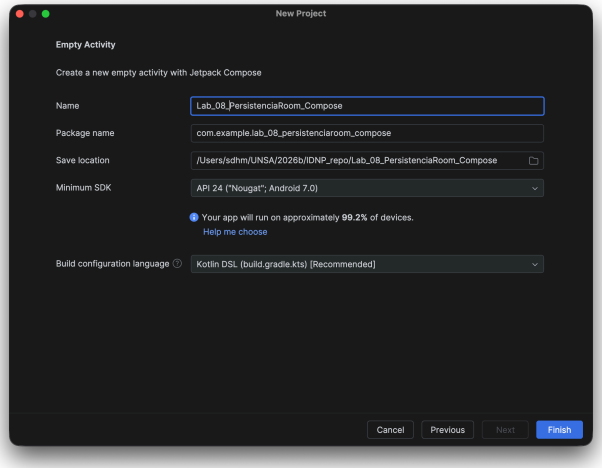


Figura 1: Paso 1: creación del proyecto PersistenciaRoom_Compose en Android Studio con Kotlin y el SDK mínimo de la plantilla.

Paso 2. Configurar Gradle para usar Room con KSP

Se modifican tres archivos, en este orden (las versiones indicadas funcionaron con Android Studio con AGP 9.3.3 y Kotlin 2.2.10; si tu proyecto usa otras versiones, revisa la nota que sigue a los códigos):

Nota (versiones y errores de sincronización). Las versiones dependen de tu Android Studio; en algunos casos hay que cambiarlas:

- Plugin [id: "com.google.devtools.ksp"] was not found: KSP está sin versión. Debe declararse con versión en libs.versions.toml y aplicarse con alias(libs.plugins.ksp) dentro de plugins { }. La línea ksp(«androidx.room:room-compiler:...») va dentro de dependencies { }.
- [ksp] IllegalStateException: unexpected jvm signature V: ocurre con Room 2.6.1 y un KSP nuevo. Usar Room 2.7.0 o superior en las tres líneas. Si Gradle no encuentra 2.7.2, probar 2.7.0 o 2.8.0.
- Con AGP 9 se necesita KSP 2.3.1 o superior (se recomienda 2.3.6). Si el Sync indica que KSP no es compatible con tu versión de Kotlin, cambiar la versión de KSP (por ejemplo 2.3.12). En proyectos con AGP 8.x, KSP usa el formato -, por ejemplo 2.2.10-2.0.2.
- Unresolved reference "compose" o "viewModel": falta lifecycle-viewmodel-compose; usar la misma versión que lifecycleRuntimeKtx del libs.versions.toml.
- Después de editar los archivos, pulsar Sync Now. Si quedan errores rojos en crearBaseDeDatos o edificioDao, ejecutar Build > Rebuild Project para que KSP genere las clases de Room.

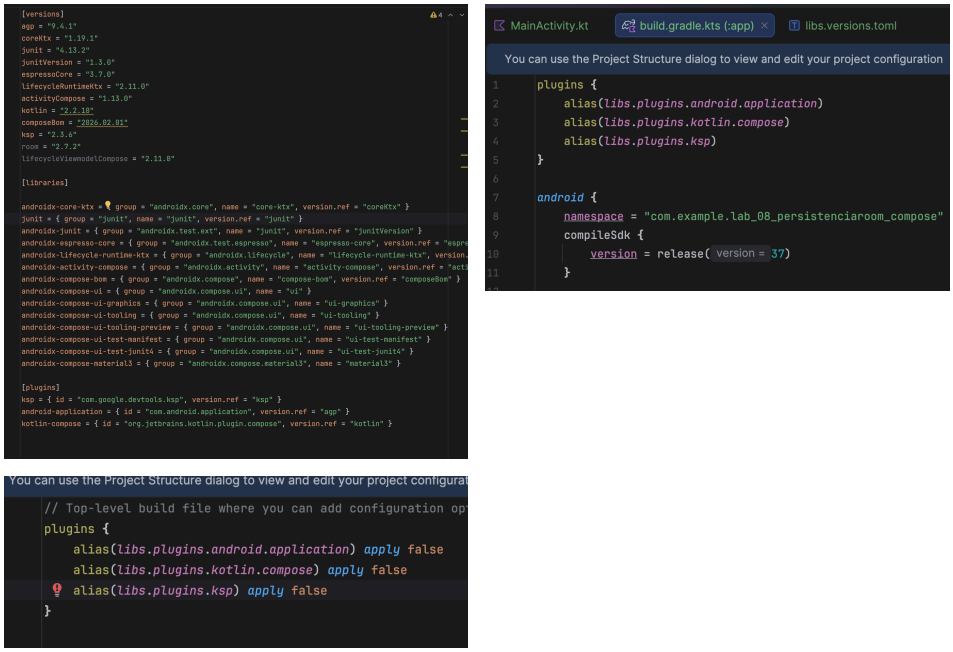


Figura 2: Paso 2: configuración de Gradle con las versiones de KSP y las dependencias de Room en libs.versions.toml, build.gradle.kts del proyecto y build.gradle.kts del módulo app, junto con el Sync Now completado sin errores.

Paso 3. Agregar las importaciones

Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio (incluye Preview, porque la plantilla del proyecto conserva Greeting y GreetingPreview; sin ese import aparece Unresolved reference "Preview"):

```
import android.content.Context
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.room.Dao
import androidx.room.Database
```

Figura 3: Paso 3: importaciones agregadas debajo del package y antes de la clase MainActivity en MainActivity.kt.

Paso 4. Definir la entidad EdificioEntity

Definir la entidad EdificioEntity:

```
31 Usages
@Entity
data class EdificioEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val nombre: String,
    val categoria: String
)
```

Figura 4: Paso 4: código de la entidad EdificioEntity con su clave primaria autogenerada, nombre y categoría.

Paso 5. Definir la entidad AmbienteEntity

Definir la entidad AmbienteEntity, agregando una ForeignKey que la relacione con EdificioEntity:

```
data class AmbienteEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val edificioId: Int,
    val nombre: String
)
```

Figura 5: Paso 5: código de la entidad AmbienteEntity con la ForeignKey que la relaciona con EdificioEntity.

Paso 6. Crear la interfaz EdificioDao

Crear la interfaz EdificioDao con las operaciones básicas de inserción, actualización, borrado y una consulta simple. Por ahora dejamos un comentario donde irá la consulta relacional:

```
7 Usages  1 Implementation
@Dao
interface EdificioDao {

    3 Usages  1 Implementation
    @Insert
    suspend fun insertarEdificio(
        edificio: EdificioEntity
    ): Long

    4 Usages  1 Implementation
    @Insert
    suspend fun insertarAmbiente(
        ambiente: AmbienteEntity
    )

    1 Usage  1 Implementation
    @Update
    suspend fun actualizarEdificio(
        edificio: EdificioEntity
    )

    1 Implementation
    @Update
    suspend fun actualizarAmbiente(
        ambiente: AmbienteEntity
    )

    1 Implementation
    @Delete
    suspend fun eliminarEdificio(
        edificio: EdificioEntity
    )

    1 Usage  1 Implementation
    @Delete
    suspend fun eliminarAmbiente(
        ambiente: AmbienteEntity
    )
```

Figura 6: Paso 6: interfaz EdificioDao con las operaciones de inserción, actualización, borrado y la consulta simple de edificios.

Paso 7. Definir EdificioConAmbientes

Definir la clase intermedia EdificioConAmbientes con `@Embedded` y `@Relation`, y agregar en el mismo Dao una consulta `@Transaction` que la devuelva. Con esto, EdificioDao queda completo:

```
7 Usages
data class EdificioConAmbientes(
    @Embedded
    val edificio: EdificioEntity,

    @Relation(
        parentColumn = "id",
        entityColumn = "edificioId"
    )
    val ambientes: List<AmbienteEntity>
)
```

Figura 7: Paso 7: clase EdificioConAmbientes con `@Embedded` y `@Relation`, y la consulta `@Transaction` que devuelve los edificios con sus ambientes.

Paso 8. Crear la clase AppDatabase

Crear la clase AppDatabase reuniendo ambas entidades y exponiendo el Dao, junto a una función que la instancie una sola vez con Room.databaseBuilder:

```
abstract class AppDatabase : RoomDatabase() {

    1 Usage   1 Implementation
    abstract fun edificioDao(): EdificioDao
}
```

Figura 8: Paso 8: clase AppDatabase con las dos entidades y la función crearBaseDeDatos que usa Room.databaseBuilder.

Paso 9. Crear el EdificiosViewModel

Crear un EdificiosViewModel que, al iniciar, inserte (si la base está vacía) 2 edificios de muestra con sus ambientes mediante una coroutine, y exponga el resultado de obtenerConAmbientes() como un StateFlow observable. Como el ViewModel necesita el Dao en su constructor, se agrega también un Factory sencillo:

```
4 Usages
class EdificiosViewModel(
    private val dao: EdificioDao
) : ViewModel() {

    2 Usages
    private val _edificios =
        MutableStateFlow<List<EdificioConAmbientes>>( value = emptyList())

    1 Usage
    val edificios: StateFlow<List<EdificioConAmbientes>> =
        _edificios

    init {
        viewModelScope.launch {
            insertarDatosIniciales()
            cargarEdificios()
        }
    }

    1 Usage
    private suspend fun insertarDatosIniciales() {
        if (dao.obtenerTodos().isEmpty()) {

            val id1 = dao.insertarEdificio(
                EdificioEntity(
                    nombre = "Biblioteca Central",
                    categoria = "Servicios"
                )
            )

            dao.insertarAmbiente(
                AmbienteEntity(
                    edificioId = id1.toInt(),
                    nombre = "Sala de lectura"
                )
            )
```

Figura 9: Paso 9: clase EdificiosViewModel con la inserción de los datos de muestra y el EdificiosViewModelFactory.

Paso 10. Crear el Composable ListaEdificiosConAmbientes

Crear el Composable ListaEdificiosConAmbientes, que observe el StateFlow del ViewModel y muestre el resultado en una LazyColumn, agrupando visualmente los ambientes bajo el nombre de cada edificio:

```
1 Usage
@Composable
fun ListaEdificiosConAmbientes(
    viewModel: EdificiosViewModel
) {
    val edificios by viewModel.edificios.collectAsState()

    Column(
        modifier = Modifier
            .fillMaxSize()
            .statusBarsPadding()
            .padding( all = 16.dp)
    ) {
        Text(
            text = "Edificios del campus",
            fontSize = 25.sp,
            fontWeight = FontWeight.Bold,
            color = MaterialTheme.colorScheme.primary
        )

        Spacer(modifier = Modifier.height(6.dp))

        Text(
            text = "Datos almacenados con Room Database",
            fontSize = 14.sp
        )

        Spacer(modifier = Modifier.height(16.dp))

        if (edificios.isEmpty()) {
            Text("Cargando edificios...")
        } else {
            LazyColumn(
```

Figura 10: Paso 10: Composable ListaEdificiosConAmbientes mostrando los edificios y sus ambientes en una LazyColumn.

Paso 11. Modificar MainActivity

Dentro de MainActivity, crear la base de datos, el Dao y el Factory, y mostrar ListaEdificiosConAmbientes dentro de setContent:

```
1 Usage
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val database = DatabaseProvider.obtener(
            applicationContext
        )

        val dao = database.edificioDao()

        val factory = EdificiosViewModelFactory(dao)

        setContent {
            Lab_08_PersistenciaRoom_ComposeTheme {
                val viewModel: EdificiosViewModel = viewModel(
                    factory = factory
                )

                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    ListaEdificiosConAmbientes(viewModel)
                }
            }
        }
    }
}
```

Figura 11: Paso 11: MainActivity completo con la creación de la base de datos, el Dao, el Factory y la llamada a ListaEdificiosConAmbientes dentro de setContent.

Paso 12. Ejecutar la aplicación

Ejecutar la aplicación en un emulador y verificar en la LazyColumn que los edificios aparecen junto con sus ambientes; cerrar y volver a abrir la aplicación y confirmar que los datos siguen ahí (persisten en la base de datos local).



Figura 12: Paso 12: aplicación en ejecución en el emulador, mostrando los edificios con sus ambientes en la LazyColumn después de cerrar y volver a abrir la aplicación.

Paso 13. Verificar la base de datos

Verificar la base de datos: Room usa SQLite y guarda el archivo app.db dentro del dispositivo. Con la app en ejecución, abrir View > Tool Windows > App Inspection > Database Inspector y revisar las tablas EdificioEntity y AmbienteEntity (deben verse 2 edificios y 3 ambientes).

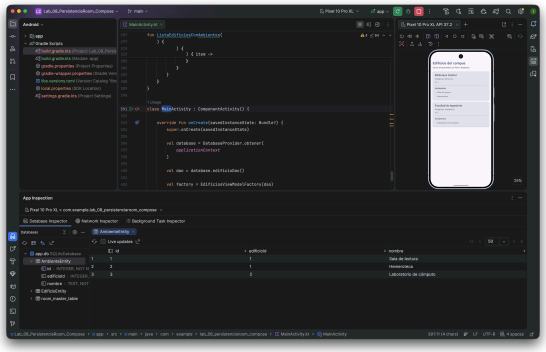


Figura 13: Paso 13: Database Inspector mostrando las tablas EdificioEntity y AmbienteEntity de la base de datos app.db, con 2 edificios y 3 ambientes.

EJERCICIOS PROPUESTOS

Amplía el modelo del ejercicio resuelto a al menos 3 tablas para almacenar la información de las edificaciones en una base de datos:

1. Diseña el diagrama del modelo de datos agregando una tercera entidad NotaEntity(id, ambienteId, texto) relacionada uno-a-muchos con AmbienteEntity (un ambiente puede tener varias notas).

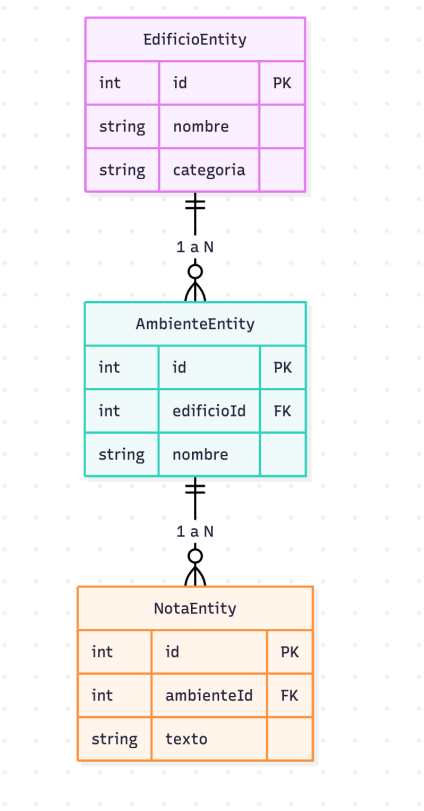


Figura 14: Diagrama del modelo de datos con la incorporación de la entidad `NotaEntity`.

¿Qué cambios se hicieron en el modelo?

1. **Incorporación de la entidad `NotaEntity`:**
   - `id`: Clave primaria (PK) autogenerada para identificar de forma unívoca cada nota.
   - `ambienteId`: Clave foránea (FK) vinculada a la clave primaria `id` de `AmbienteEntity`.
   - `texto`: Campo de tipo texto para el contenido de la nota.
2. **Definición de la relación uno a muchos (1:N):**
   - `AmbienteEntity` → `NotaEntity`: Un ambiente puede tener asociadas cero o múltiples notas (`||--o{`), mientras que cada registro en `NotaEntity` pertenece obligatoriamente a un único ambiente mediante su clave foránea `ambienteId`.

2. Implementa el modelo completo con Room: las 3 entidades, sus claves foráneas, y el Dao correspondiente con @Insert, @Update y @Delete para las 3 tablas.

1. Definición de la entidad `NotaEntity`

Se implementa la tercera entidad vinculada a `AmbienteEntity` mediante una clave foránea (`ForeignKey`):

```
@Entity(
    foreignKeys = [
        ForeignKey(
            entity = AmbienteEntity::class,
```

```
            parentColumns = ["id"],
            childColumns = ["ambienteId"]
        )
    ]
)
data class NotaEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val ambienteId: Int,
    val texto: String
)
```

2. Operaciones CRUD en `EdificioDao`

Se incorporan los métodos de inserción, actualización y eliminación para las tres tablas:

```
@Dao
interface EdificioDao {
    // --- INSERT (3 tablas) ---
    @Insert suspend fun insertarEdificio(edificio: EdificioEntity): Long
    @Insert suspend fun insertarAmbiente(ambiente: AmbienteEntity)
    @Insert suspend fun insertarNota(nota: NotaEntity): Long

    // --- UPDATE (3 tablas) ---
    @Update suspend fun actualizarEdificio(edificio: EdificioEntity)
    @Update suspend fun actualizarAmbiente(ambiente: AmbienteEntity)
    @Update suspend fun actualizarNota(nota: NotaEntity)

    // --- DELETE (3 tablas) ---
    @Delete suspend fun eliminarEdificio(edificio: EdificioEntity)
    @Delete suspend fun eliminarAmbiente(ambiente: AmbienteEntity)
    @Delete suspend fun eliminarNota(nota: NotaEntity)
}
```

3. Registro en `AppDatabase`

Se incorpora `NotaEntity` al arreglo de entidades de Room:

```
@Database(
    entities = [
        EdificioEntity::class,
        AmbienteEntity::class,
        NotaEntity::class // <-- Agregada aquí
    ],
    version = 1,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun edificioDao(): EdificioDao
}
```

Resumen de aspectos clave

- **Entidad `NotaEntity`:** Incorpora clave primaria autogenerada (`id`) y clave foránea (`ambienteId` → `AmbienteEntity.id`).
- **DAO unificado:** Expone operaciones CRUD completas (`@Insert`, `@Update`, `@Delete`) para `EdificioEntity`, `AmbienteEntity` y `NotaEntity`.
- **Esquema Room:** Inclusión de `NotaEntity::class` dentro del arreglo `entities` de `AppDatabase`.

3. Realiza una segunda consulta relacional (por ejemplo, obtener un AmbienteEntity junto con su List mediante @Embedded/@Relation), distinta a la implementada en el ejercicio resuelto

1. Clase de relación POJO: `AmbienteConNotas`

Se define la clase intermedia para modelar el vínculo 1 a muchos entre `AmbienteEntity` y `NotaEntity`:

```
data class AmbienteConNotas(
    @Embedded
    val ambiente: AmbienteEntity,

    @Relation(
        parentColumn = "id",
        entityColumn = "ambienteId"
    )
    val notas: List<NotaEntity>
)
```

- **`@Embedded`:** Incrusta la entidad padre `AmbienteEntity`.
- **`@Relation`:** Establece el enlace 1 a muchos conectando la columna primaria `id` de `AmbienteEntity` con la clave foránea `ambienteId` de `NotaEntity`.

2. Método de consulta en el DAO: `EdificioDao`

Se agrega el método relacional con transacción atómica garantizada:

```
@Transaction
@Query("SELECT * FROM AmbienteEntity")
suspend fun obtenerAmbientesConNotas(): List<AmbienteConNotas>
```

- **`@Transaction`:** Garantiza la consistencia atómica de la lectura para que Room ejecute la consulta del padre y las consultas de relación hijas en una sola transacción.
- **`@Query`:** Consulta la tabla `AmbienteEntity` y Room rellena automáticamente la lista `notas` asociada a cada ambiente.

4. Agrega a la interfaz botones para eliminar y actualizar un edificio o ambiente existente,verificando que los cambios persistan.

Se implementó la funcionalidad completa de actualización y eliminación para edificios y ambientes con un diseño minimalista en Jetpack Compose, asegurando la persistencia y consistencia en SQLite mediante Room.

1. Configuración de persistencia e integridad referencial

- **Eliminación en cascada (`ForeignKey.CASCADE`):** Se configuró `onDelete = ForeignKey.CASCADE` en las anotaciones `@ForeignKey` de `AmbienteEntity` y `NotaEntity`. Al eliminar un edificio, SQLite remueve automáticamente sus ambientes asociados, preservando la integridad referencial.
- **Dependencias de UI:** Se agregó `androidx.compose.material:material-icons-core` para disponer de los iconos vectoriales `Icons.Default.Edit` e `Icons.Default.Delete`.

2. Métodos CRUD en `EdificiosViewModel`

Se implementaron funciones asíncronas en el ViewModel que ejecutan las operaciones en segundo plano mediante corrutinas y recargan inmediatamente el flujo observable `StateFlow`:

```
fun actualizarEdificio(edificio: EdificioEntity) {
    viewModelScope.launch { dao.actualizarEdificio(edificio); cargarEdificios() }
}

fun eliminarEdificio(edificio: EdificioEntity) {
    viewModelScope.launch { dao.eliminarEdificio(edificio); cargarEdificios() }
```

```
}

fun actualizarAmbiente(ambiente: AmbienteEntity) {
    viewModelScope.launch { dao.actualizarAmbiente(ambiente); cargarEdificios() }
}

fun eliminarAmbiente(ambiente: AmbienteEntity) {
    viewModelScope.launch { dao.eliminarAmbiente(ambiente); cargarEdificios() }
}
```

3. Botones de acción en la interfaz (Compose)

Dentro de cada tarjeta `OutlinedCard` de edificio y fila de ambiente se integraron botones interactivos (`IconButton`):

```
// Acciones para el Edificio
IconButton(onClick = { edificioAEditar = item.edificio }) {
    Icon(Icons.Default.Edit, contentDescription = "Editar", tint =
MaterialTheme.colorScheme.primary)
}
IconButton(onClick = { edificioAEliminar = item.edificio }) {
    Icon(Icons.Default.Delete, contentDescription = "Eliminar", tint =
MaterialTheme.colorScheme.error)
}

// Acciones para cada Ambiente
IconButton(onClick = { ambienteAEditar = ambiente }) {
    Icon(Icons.Default.Edit, contentDescription = "Editar")
}
IconButton(onClick = { ambienteAEliminar = ambiente }) {
    Icon(Icons.Default.Delete, contentDescription = "Eliminar", tint =
MaterialTheme.colorScheme.error)
}
```

4. Diálogos modales de edición y confirmación (`AlertDialog`)

La interacción con el usuario se gestiona mediante diálogos de Material 3:

- **Diálogo de edición:** Despliega campos de texto (`OutlinedTextField`) para modificar nombres o categorías, invocando `viewModel.actualizarEdificio(...)` o `viewModel.actualizarAmbiente(...)` al confirmar.
- **Diálogo de eliminación:** Solicita confirmación explícita (advirtiendo sobre la cascada en edificios) y ejecuta `viewModel.eliminarEdificio(...)` o `viewModel.eliminarAmbiente(...)`.

Al confirmar cualquier modificación, Room ejecuta las sentencias SQL (`@Update` o `@Delete`), actualizando de forma reactiva la interfaz sin necesidad de recargar la aplicación manualmente.

5. Verificación y pruebas de persistencia en ejecución

A continuación se evidencia la secuencia de validación en el emulador Android, comprobando la interacción con los controles de edición, la confirmación de cambios y la persistencia en Room tras cerrar la aplicación:



(a) Botones de acción **(b)** Diálogo de edición **(c)** Cierre de la app **(d)** Persistencia validada

Figura 15: Secuencia de verificación de actualización y persistencia en Room: (a) Interfaz con botones de editar y eliminar; (b) Diálogo para renombrar a «Sala de lectura V1»; (c) Salida al launcher para forzar el cierre; (d) Reapertura de la aplicación con los cambios persistidos.

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (datos de edificios y ambientes almacenados con Room, que persisten al cerrar y volver a abrir la app)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Respuesta:

Sí, se completó satisfactoriamente el ejercicio resuelto propuesto por el docente. La arquitectura implementa Room sobre SQLite (`app.db`), persistiendo de forma estructurada las entidades `EdificioEntity` y `AmbienteEntity` vinculadas mediante clave foránea con integridad referencial.

Al ejecutar la aplicación por primera vez, el `EdificiosViewModel` realiza la inserción inicial de los edificios y ambientes de prueba dentro de una corrutina en `viewModelScope`. La interfaz desarrollada en Jetpack Compose observa el flujo `StateFlow` expuesto por el método relacional `obtenerConAmbientes()` para renderizar los datos organizados en una `LazyColumn`.

Para validar la persistencia:

1. Se ejecutó la aplicación y se verificó la correcta carga de los registros en la interfaz.
2. Se cerró completamente el proceso de la aplicación desde la vista multitarea del emulador (forzando su detención).
3. Al volver a abrir la aplicación desde el iniciador de Android, los datos se recuperaron íntegramente desde el almacenamiento local SQLite sin pérdida de información ni duplicación de registros.



Figura 16: Evidencia de ejecución en el emulador Android tras cerrar y reiniciar la aplicación, confirmando la persistencia de los registros en Room.

2. ¿Qué otra alternativa de almacenamiento (además de Room) se podría emplear en una aplicación con necesidades similares? Justifica tu respuesta comparando ventajas y desventajas frente a Room.

Respuesta:

Para una aplicación con necesidades relacionales (persistencia de entidades vinculadas 1:N, operaciones CRUD e integración reactiva con Jetpack Compose), las principales alternativas en el ecosistema Android son:

1. SQLDelight

Es la alternativa relacional más destacada frente a Room en Kotlin:

- **Enfoque:** A diferencia de Room (donde las anotaciones Kotlin generan el esquema SQL), en SQLDelight se escribe SQL estándar y el plugin genera código Kotlin fuertemente tipado con verificación en tiempo de compilación.
- **Ventajas frente a Room:** Soporte nativo para Kotlin Multiplatform (KMP), permitiendo reutilizar el 100% de la capa de datos en Android, iOS, Desktop y Backend; control absoluto y transparente sobre las sentencias SQL ejecutadas.
- **Desventajas frente a Room:** Carece de abstracciones automáticas de conveniencia como las anotaciones relacionales `\@Embedded` y `\@Relation` de Room; menor ecosistema de extensiones directas dentro de Google Jetpack.

2. Realm / Atlas Device SDK (Base de datos NoSQL orientada a objetos)

- **Enfoque:** Motor de persistencia orientado a objetos que almacena instancias directamente sin necesidad de mapeo objeto--relacional (ORM).
- **Ventajas frente a Room:** Alto rendimiento en transacciones y consultas sobre grafos de objetos complejos; sincronización reactiva integrada con la UI sin pasar por SQLite.
- **Desventajas frente a Room:** Incrementa sensiblemente el tamaño final del APK debido a sus bibliotecas nativas C++; menor flexibilidad si se requieren consultas analíticas complejas o migraciones estándar SQL.

3. Tabla comparativa frente a Room

Mecanismo	Ventajas frente a Room	Desventajas frente a Room
SQLDelight	Multiplataforma nativo (KMP), tipado estricto a partir de SQL puro, sin overhead de KSP.	Requiere escribir todo el SQL a mano; sin anotaciones automáticas 1:N.
Realm (Atlas)	Elimina el mapeo relacional (ORM); consultas orientadas a objetos muy veloces.	Binarios C++ pesados en el APK; dependencia de proveedor externo.
DataStore (Proto)	Transaccional, liviano, sin overhead de SQLite para configuraciones.	No soporta consultas relacionales, claves foráneas ni indexación relacional.

Conclusión de selección: Para desarrollo exclusivo en Android con Jetpack Compose, **Room** sigue siendo la opción óptima por su soporte oficial en AndroidX, estabilidad de migraciones y verificación estática de consultas. Si el proyecto requiriera evolucionar hacia Kotlin Multiplatform (compartiendo lógica con iOS), **SQLDelight** sería el reemplazo idóneo.

3. ¿En dónde almacena Room los datos de la aplicación (almacenamiento interno o externo del dispositivo)? Justifica tu respuesta y explica qué implica para la privacidad de los datos del usuario.

Respuesta:

Room almacena las bases de datos en el **almacenamiento interno privado** del dispositivo (`Internal Storage`), dentro del directorio sandbox asignado exclusivamente a la aplicación por el sistema operativo Android:

$$/\text{data}/\text{data}/<\text{package_name}>/\text{databases}/<\text{database_name}>.\text{db}$$

Para este proyecto, la ruta concreta en el sistema de archivos corresponde a: `/data/data/com.example.persistenciaroom_compose/databases/app.db` (acompañada por sus archivos de registro en modo WAL: `app.db-wal` y `app.db-shm`).

1. Justificación técnica

- **Comportamiento de `Room.databaseBuilder`:** Al instanciar la base de datos con `Room.databaseBuilder(context, AppDatabase::class.java, "app.db")`, el framework delega la resolución del archivo al método del framework `context.getDatabasePath("app.db")`, el cual apunta intrínsecamente al almacenamiento interno privado de la app.
- **Modelo de aislamiento de Android (Sandbox de Linux):** En el sistema operativo Android, cada aplicación se ejecuta con un UID (User ID) único dentro del kernel de Linux. El directorio `/data/data/<package_name>` se crea con permisos Unix estrictos (`rwx------`), impidiendo que procesos con otro identificador de usuario puedan listar, leer o escribir en sus archivos.

2. Implicaciones para la privacidad de los datos del usuario

1. **Aislamiento y confidencialidad frente a terceros:** Ninguna otra aplicación instalada en el dispositivo puede acceder ni leer la base de datos `app.db`, resguardando la información sin necesidad de solicitar permisos especiales en tiempo de ejecución (`Runtime Permissions`).
2. **Ciclo de vida sincronizado con la aplicación:** Al desinstalar la aplicación o limpiar su almacenamiento desde *Ajustes* > *Aplicaciones*, el sistema operativo purga automáticamente el directorio privado, garantizando que no queden datos residuales huérfanos.
3. **Inaccesibilidad en dispositivos no modificados:** Los datos permanecen ocultos ante exploradores de archivos estándar; únicamente son accesibles en entornos de depuración autorizados mediante ADB (`Database Inspector` de Android Studio) o en dispositivos que cuenten con acceso `root`.
4. **Compatibilidad con cifrado en reposo:** Si se requiere proteger la información frente a extracciones forenses físicas en dispositivos vulnerados, Room se integra directamente con bibliotecas como **SQLCipher**, cifrando el archivo SQLite en almacenamiento interno mediante el algoritmo AES-256.

III. CONCLUSIONES

- **Persistencia reactiva y sincronización fluida:** Integrar Room con Jetpack Compose y corrutinas nos enseña que la base de datos local funciona mejor como una «fuente única de verdad» viva. Al transformar los datos en flujos observables (`StateFlow`), la interfaz se actualiza de manera automática cada vez que ocurre un cambio, eliminando la necesidad de recargas manuales y evitando bloqueos en la pantalla.
- **Confiar la integridad al motor de base de datos:** El uso de claves foráneas con eliminación en cascada (`CASCADE`) demuestra lo valioso que es delegar las reglas de consistencia a SQLite en lugar de programar validaciones manuales en el código. Esto previene que queden registros huérfanos (como notas sin ambiente) y simplifica el mantenimiento a medida que el sistema crece.
- **Prevención de errores desde la compilación:** La verificación que realiza Room junto a KSP en tiempo de compilación aporta una gran tranquilidad durante el desarrollo. Detectar errores de sintaxis SQL o tipos incompatibles antes de ejecutar la aplicación ahorra tiempo de depuración y da confianza para refactorizar o agregar nuevas tablas sin temor a fallos inesperados.
- **Modelado relacional natural para la interfaz:** Diseñar la jerarquía de edificios, ambientes y notas mediante clases intermedias (`@Relation`) evidencia que un buen diseño relacional repercute directamente en la comodidad del desarrollador. La interfaz puede consumir estructuras completas y ordenadas sin necesidad de realizar múltiples consultas dispersas ni transformar datos a mano.
- **Privacidad protegida por diseño en Android:** Comprender que la base de datos reside en el almacenamiento interno privado (`sandbox`) resalta la importancia de la seguridad por defecto. Saber que los datos quedan aislados de otras aplicaciones y se eliminan limpiamente al desinstalar la app nos recuerda que una buena arquitectura debe cuidar la privacidad del usuario desde la capa de persistencia.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.